

Order Management System

(Dual Integration Approach)

Repository: <https://github.com/mariyappan2111/kpn-order-management-system.git>

Open API:

https://virtserver.swaggerhub.com/go2clouds/Orders/1.0.0/orders?customerId=cust_existing_12345&status=confirmed&limit=20&offset=0

<https://virtserver.swaggerhub.com/go2clouds/product/1.0.0/products?active=true>

Summary

This document explains a comprehensive Order Management solution for Salesforce that provides flexibility in data sourcing and order processing through dual integration pathways. The solution enables users to manage orders without leaving the order detail page, supporting both native Salesforce operations and external API integrations.

Architecture Overview

Solution Design:

The implementation follows a **hybrid architecture** pattern that supports:

1. **Native Salesforce Mode:** Utilizes standard Salesforce objects (Product2, PricebookEntry, Order, OrderItem)
2. **External API Mode:** Integrates with external product and order management systems
3. **Seamless Switching:** Business logic abstracted to allow runtime configuration between modes

Component Structure

Lightning Web Components (LWC)

availableProducts (Product Selection)

orderProducts (Orders)

Apex Controllers (Service Layer)

KPN_ProductController(Salesforce)

KPN_ProductsServiceController (External Product API)

KPN_OrderController (Salesforce Order Operations)

KPN_OrderServiceController (External Order API)

Apex Utilities

KPN_IntegrationsBase (HTTP Callout Framework)

KPN_ProductWrapper (Product Data Model)

KPN_OrderItemWrapper (Order Item Data Model)

Test Classes (80%+ Coverage)

KPN_ProductControllerTest

KPN_OrderControllerTest

KPN_IntegrationsBase_Test

KPN_OrderServiceController_Test

KPN_MockHttpResponse (Mock Callout Handler)

Component Details

1. Available Products Component

Purpose: Displays products available for ordering based on the Order's associated Pricebook.

Key Features:

- **Dual Data Source Support:**
 - **Salesforce Mode:** Queries Product2 and PricebookEntry objects filtered by Order's Pricebook2Id
 - **API Mode:** Calls external Product API to retrieve product catalog
- **Smart Product Sorting:** Products already on the order appear at the top of the list
- **Duplicate Prevention:** Each product appears only once in the available products list
- **Two-Column Display:** Shows Product Name and List Price in a clean, scannable format
- **Add to Order Functionality:**
 - New products are added with quantity = 1
 - Existing products have their quantity incremented by 1

Technical Implementation:

- LWC communicates with Apex via @AuraEnabled methods
- Component uses @wire for reactive data fetching
- Lightning Message Service for inter-component communication

2. Order Products Component

Purpose: Displays and manages products already added to the current order.

Note: there is already two Products is displayed in the order table(API Order)

Key Features:

- **Comprehensive Order Display:** Shows Name, Unit Price, Quantity, and Total Price in a data table
- **Real-time Updates:** Automatically refreshes when products are added via Available Products component
- **Order Activation:** "Activate" button sets **Order.Status = 'Activated'** and prevents further modifications
- **State Management:** Disables add/edit actions post-activation
- **Dual Persistence:**
 - **API Mode:** Sends order data to external Order API.
 - **Salesforce Mode:** Creates/updates OrderItem records

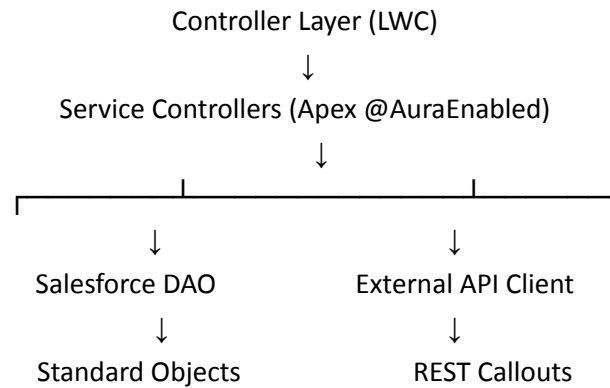
Technical Implementation:

- Lightning-datatable for tabular display
- Computed properties for Total Price calculation
- Event handling for cross-component communication
- Conditional rendering based on Order status

Apex Service Layer Architecture

Pattern: Strategy Pattern for Data Source Abstraction

The solution implements a strategy pattern allowing business logic to remain consistent while swapping data sources:



Key Classes:

KPN_ProductController

- **Purpose:** Salesforce native product operations
- **Methods:**
 - **getAvailableProducts(orderId):** Queries products from Pricebook
 - Filters by Order's Pricebook2Id
 - Returns products not yet on order + existing order products

KPN_ProductsServiceController

- **Purpose:** External API product integration
- **Methods:**
 - **getProductsFromAPI(orderId):** Fetches products via HTTP callout
 - Parses JSON response into **KPN_ProductWrapper** objects
 - Merges with existing order products for comprehensive view

KPN_OrderController

- **Purpose:** Salesforce native order operations
- **Methods:**
 - **addProductToOrder(orderId, productId):** Creates/updates OrderItem
 - **getOrderProducts(orderId):** Retrieves existing order lines
 - **activateOrder(orderId):** Sets Order and OrderItems to 'Activated'
 - Implements quantity increment logic for duplicate products

KPN_OrderServiceController

- **Purpose:** External API order integration
- **Methods:**
 - **createOrderViaAPI(orderData):** Posts order to external system
 - Handles order payload serialization
 - Returns confirmation or error messages

KPN_IntegrationsBase

- **Purpose:** Reusable HTTP callout framework
- **Features:**
 - Named Credential management
 - Request/response logging
 - Error handling and retry logic
 - Supports GET, POST, PUT, DELETE methods

Data Models

KPN_ProductWrapper

```
public with sharing class KPN_ProductWrapper {
    public KPN_ProductWrapper() { }
    @AuraEnabled public String productName;
    @AuraEnabled public String productId;
    @AuraEnabled public String pricebookEntryId;
    @AuraEnabled public Decimal listPrice;
    @AuraEnabled public Boolean isInOrder;
    @AuraEnabled public Decimal quantityOnOrder;
    @AuraEnabled public String id;
    @AuraEnabled public String name;
    @AuraEnabled public String category;
    @AuraEnabled public String description;
    @AuraEnabled public Decimal price;
    @AuraEnabled public String currencyType;
    @AuraEnabled public Boolean isBundle;

    public KPN_ProductWrapper(Map<String, Object> data) {
        productId = (String) data.get('id');
        id = (String) data.get('id');
        productName = (String) data.get('name');
        category = (String) data.get('category');
        description = (String) data.get('description');
        listPrice = (data.containsKey('price')) ? Decimal.valueOf(String.valueOf(data.get('price'))) : 0;
        currencyType = (String) data.get('currencytype');
        isBundle = (Boolean) data.get('isBundle');
    }
}
```

KPN_OrderItemWrapper

```
public with sharing class KPN_OrderItemWrapper {

    public class PhoneNumber {
        public String countryCode;
        public String phnumber;
        public String extension;
    }

    public class Address {
        public String addressLine1;
        public String addressLine2;
        public String city;
        public String stateProvince;
        public String postalCode;
        public String country;
        public String deliveryInstructions;
    }

    public class Customer {
        public String customerId;
        public String firstName;
        public String lastName;
        public String email;
        public PhoneNumber phone;
        public PhoneNumber alternatePhone;
        public Date dateOfBirth;
        public String identificationType;
        public String identificationNumber;
    }

    public virtual class OrderItem {
        public String productId;
        public Integer quantity;
        public Map<String, Object> customizations;
    }

    public class OrderItemDetail extends OrderItem {
        public String productName;
        public Decimal unitPrice;
        public Decimal totalPrice;
    }

    public class PriceBreakdown {
```

```
    public Decimal subtotal;
    public Decimal setupFees;
    public Decimal taxes;
    public Decimal discounts;
    public Decimal shippingFee;
    public Decimal total;
}

public class Pagnation {
    public Integer total;
    public Integer pglimit;
    public Integer offset;
}

public class OrderSummary {
    public String orderId;
    public String orderNumber;
    public String status;
    public String customerId;
    public String customerName;
    public Decimal totalAmount;
    public Datetime orderDate;
    public Date scheduledDeliveryDate;
}

public class CreateOrderRequest {
    public Customer customer;
    public List<OrderItem> orderItems;
    public Address deliveryAddress;
    public Address billingAddress;
    public Date preferredDeliveryDate;
    public String preferredTimeSlot;
    public Boolean installationRequired;
    public String notes;
}

public class OrderResponse {
    public String orderId;
    public String orderNumber;
    public String status;
    public Customer customer;
    public List<OrderItemDetail> orderItems;
    public Address deliveryAddress;
    public Address billingAddress;
    public Date scheduledDeliveryDate;
    public String deliveryTimeSlot;
```

```

    public Boolean installationScheduled;
    public Datetime installationDateTime;
    public Decimal totalAmount;
    public PriceBreakdown breakdown;
    public String trackingNumber;
    public Date estimatedActivationDate;
    public Datetime createdAt;
    public Datetime updatedAt;
}

public class OrderListResponse {
    public List<OrderSummary> orders;
    public Pagination pagination;
}

public class ErrorResponse {
    public String code;
    public String message;
    public List<ErrorDetail> details;
}

public class ErrorDetail {
    public String field;
    public String issue;
}

public static OrderListResponse parseListResponse(String jsonStr) {
    return (OrderListResponse) JSON.deserialize(jsonStr, OrderListResponse.class);
}

public static OrderResponse parseOrderResponse(String jsonStr) {
    return (OrderResponse) JSON.deserialize(jsonStr, OrderResponse.class);
}
}

```

OpenAPI Specifications

1. Product API Specification

Purpose: Retrieve product catalog suitable for telecom/subscription services

Service End URL: <https://virtserver.swaggerhub.com/go2clouds/product/1.0.0/products?active=true>

Key Endpoints:

GET /api/v1/products

Parameters:

- pricebookId (query): Filter by pricebook

- category (query): Filter by product category

Response:

- 200: Array of Product objects
- 404: No products found

GET /api/v1/products/{productId}

Response:

- 200: Detailed product with bundles
- 404: Product not found

Product Object Model:

- Supports **product bundles** (e.g., Internet Plan with Mobile, Wi-Fi, TV)
- Includes pricing tiers, contract terms, availability
- Supports add-on services

2. Order API Specification

Purpose: Create orders with complete customer and delivery information.

Service End URL:

GET : ORDERS:

https://virtserver.swaggerhub.com/go2clouds/Orders/1.0.0/orders?customerId=cust_existing_12345&status=confirmed&limit=20&offset=0

GET: ORDERS BY ID

https://virtserver.swaggerhub.com/go2clouds/Orders/1.0.0/orders/ord_2025_123457

POST: Create New Order

<https://virtserver.swaggerhub.com/go2clouds/Orders/1.0.0/orders>

Request Body:

```
{
  "customer": {
    "customerId": "cust_existing_12345",
    "firstName": "Sarah",
    "lastName": "Johnson",
    "email": "sarah.j@example.com",
    "phone": {
      "countryCode": "+1",
      "number": "5552223333"
    },
    "alternatePhone": {
      "countryCode": "+1",
      "number": "5554445555"
    },
    "dateOfBirth": "1990-03-22",
    "identificationType": "passport",
```



```
"identificationNumber": "P9876543"
},
"orderItems": [
  {
    "productId": "prod_fiber_1gbps_bundle",
    "quantity": 1,
    "customizations": {
      "staticIPAddress": true,
      "managedWiFi": true
    }
  },
  {
    "productId": "prod_mobile_unlimited_5g",
    "quantity": 3,
    "customizations": {
      "phoneNumbers": [
        "5551111111",
        "5552222222",
        "5553333333"
      ],
      "numberPorting": true,
      "previousCarrier": "OldTelecom Inc",
      "accountNumber": "ACC-9876543"
    }
  },
  {
    "productId": "prod_addon_mesh_extender",
    "quantity": 2
  },
  {
    "productId": "prod_addon_streaming_premium",
    "quantity": 1,
    "customizations": {
      "serviceTier": "premium",
      "autoRenew": true
    }
  }
],
"deliveryAddress": {
  "addressLine1": "456 Oak Avenue",
  "city": "Portland",
  "stateProvince": "OR",
  "postalCode": "97201",
  "country": "US",
  "deliveryInstructions": "Leave with front desk if not home. Building code: 1234"
},
```

```
"billingAddress": {
  "addressLine1": "PO Box 789",
  "city": "Portland",
  "stateProvince": "OR",
  "postalCode": "97202",
  "country": "US"
},
"preferredDeliveryDate": "2025-11-20",
"preferredTimeSlot": "afternoon",
"installationRequired": true,
"notes": "Upgrading from existing service. Keep same account number if possible."
}
```

Key Endpoints:

POST /api/v1/orders

Request Body:

- customerId
- products[] (productId, quantity, category, unitPrice)
- deliveryAddress
- billingAddress
- contactPreferences

Response:

- 201: Order created with orderId
- 400: Validation errors

GET /api/v1/orders/{orderId}

Response:

- 200: Full order details
- 404: Order not found

Deployment Guide

Prerequisites

1. **Salesforce Developer Org** (or Sandbox/Production)
2. **Salesforce CLI (sf)** installed
3. **Git** repository access
4. **VS Code** with Salesforce Extensions (recommended)

Step-by-Step Deployment

1. Authenticate with Salesforce Org

Authenticate to your org

```
sf org login web --alias MyDevOrg --instance-url https://login.salesforce.com
```

Set as default org

```
sf config set target-org=MyDevOrg
```

2. Clone Repository

```
git clone https://github.com/mariyappan2111/kpn-order-management-system.git
```

```
cd <project directory name>
```

3. Deploy Metadata

Deploy all components

```
sf project deploy start --manifest manifest/package.xml
```

Or deploy specific components

```
sf project deploy start --source-dir force-app/main/default
```

4. Configure Named Credentials

Manual Steps Required:

1. Navigate to Setup → Named Credentials
2. Configure **Open_ExternalAPI** with:
 - External API endpoint URL - <https://virtserver.swaggerhub.com/go2clouds>
 - Authentication - **No Authentication**
 - API credentials - **None**

5. Configure Remote Site Settings

Already included in package.xml: **SwaggerHub**

URL: <https://virtserver.swaggerhub.com>

Verify in Setup → Remote Site Settings

6. Assign Permission Sets

Assign permission set to users

```
sf org assign permset --name Apex_Class_Permissions
```

Or

Setup → Permission Sets → Apex_Class_Permissions → Manage Assignments

7. Add Components to Order Record Page

Option A: Via Lightning App Builder

1. Navigate to any Order record
2. Click Setup Gear → Edit Page
3. Drag **availableProducts** component to desired location
4. Drag **orderProducts** component below it

5. Save and Activate

Option B: Deploy Pre-configured Flexipage

Already included: Order_Record_Page

Activate via Setup → Lightning App Builder → Order_Record_Page → Activation

8. Create Test Data

Execute Anonymous Apex to create sample data

Minimum Required Data:

- Standard Pricebook with Products
- PricebookEntries linking Products to Pricebook
- At least one Order with Pricebook assigned

Production Readiness Checklist

Code Quality

- 80%+ test coverage achieved - Done.
- Bulk-safe SOQL (no queries in loops) - Done.
- Proper exception handling - Done.
- Governor limit considerations - Done.
- Input validation and sanitization - Done.

Performance Optimization

- **Large Dataset Handling:**
 - Pagination in LWC datatables.
 - Indexed fields for queries (Pricebook2Id, OrderId)
 - Lazy loading for product lists
- **Caching Strategy:**
 - @wire with Lightning Data Service
 - Reduce redundant callouts

Security

- Named Credentials for external APIs (no hardcoded credentials).
- Permission Sets for controlled access.
- with sharing keywords in Apex classes.
- Lightning Locker Service compliance in LWC.

Testing Strategy

Unit Tests (Apex)

- **Positive Scenarios:** Valid product additions, order activation
- **Negative Scenarios:** Duplicate products, invalid IDs, null parameters
- **Bulk Testing:** 200+ records in single transaction

- **Mock Callouts:** `KPN_MockHttpResponse` for API simulation

Integration Tests

- **Salesforce Mode:** End-to-end with real Order/Product objects
- **API Mode:** Mocked HTTP responses for external systems

User Acceptance Testing (UAT)

- Test scripts covering all acceptance criteria
- Video demonstration of functionality (included in repo)

Best Practices Implemented

1. Separation of Concerns

- **LWC:** Presentation layer only
- **Apex Controllers:** Business logic
- **Wrapper Classes:** Data transfer objects

2. Component Independence

- No hard dependencies between `availableProducts` and `orderProducts`
- Communication via Lightning Message Service or custom events
- Can be placed anywhere on page layout

3. Error Handling

// LWC: User-friendly error messages

```
import { ShowToastEvent } from 'lightning/platformShowToastEvent';
```

```
catch (error) {  
  this.dispatchEvent(new ShowToastEvent({  
    title: 'Error adding product',  
    message: error.body.message,  
    variant: 'error'  
  }));  
}
```

4. Reusability

- `KPN_IntegrationsBase` can be extended for other API integrations
- Wrapper classes standardize data format across modes

Troubleshooting Guide

Common Issues

1. Products Not Displaying

Cause: Order missing Pricebook2Id **Solution:** Ensure Order has a Pricebook assigned before opening page

2. API Callouts Failing

Cause: Named Credential misconfigured **Solution:** Verify endpoint URL and authentication in Setup → Named Credentials

3. Permission Errors

Cause: User lacks object access **Solution:** Assign `Apex_Class_Permissions` permission set

4. Test Failures

Cause: Missing test data setup **Solution:** Review `@testSetup` methods for complete data creation

Version Control

- Use gitHub versioning (e.g., v1.2.0)
- Tag releases in Git

Conclusion

This solution provides a **production-ready, flexible Order Management system** that supports both native Salesforce operations and external API integrations. The dual-mode architecture ensures:

- **Business Continuity:** Fallback to Salesforce if external API unavailable
- **Scalability:** Handles large product catalogs and order volumes
- **Maintainability:** Clean separation of concerns and comprehensive testing
- **Extensibility:** Easy to add new features or integrate additional systems